Approximation and Pytorch Implementation of the Poisson Binomial Distribution

KAN Chun Yu Matthew

Department of Statistics and Data Science
The Chinese University of Hong Kong

8th April 2026

Abstract

The Poisson Binomial Distribution generalizes the classical binomial distribution to model the sum of independent but non-identically distributed Bernoulli trials, with applications spanning hypothesis testing, statistical quality control, and machine learning. Despite its significance, PyTorch's distributions module lacks native support for this distribution. This project implements a production-ready 'PoissonBinomial' class for PyTorch, providing efficient computation of probability mass functions, cumulative distribution functions, sampling, and statistical properties. The implementation integrates multiple computational back-ends, including Discrete Fourier Transform methods for $O(n \log n)$ exact computation and Refined Normal Approximations methods for scalable inference. Key features include strict adherence to PyTorch's distribution API standards, GPU-accelerated batch processing, automatic algorithm selection based on numerical stability, and comprehensive support for statistical inference, including mean, variance, skewness, and quantile computations. Extensive testing ensures numerical accuracy and gradient correctness, while thorough documentation and benchmarks facilitate integration with existing PyTorch probabilistic programming frameworks. This contribution fills a critical gap in PyTorch's probabilistic modeling toolkit, enabling practitioners to incorporate heterogeneous Bernoulli processes into modern deep learning workflows seamlessly.

Keywords: Poisson Binomial Distribution, PyTorch, Discrete Fourier Transform, Refined Normal Approximations

1. Introduction

The Poisson Binomial Distribution (PBD) is a generalization of the binomial distribution, representing the sum of n independent but not necessarily identically distributed

Bernoulli trials. PBD has wide applications in statistics, machine learning, and data science. These include hypothesis testing in multiple testing scenarios and false discovery rate control, statistical quality control for modeling acceptance sampling with varying defect rates, and uncertainty quantification in ensemble machine learning models that combine heterogeneous base learners. However, the direct computation of this PMF has an exponential complexity $O(2^n)$, making it computationally prohibitive for large values of n and rendering it challenging to study or apply in practical and complex scenarios. To address this issue, the Discrete Fourier Transform[1] (DFT) algorithm is used to compute the exact PBD efficiently, while the Refined Normal Approximation[2] (RNA) algorithm provides a fast and accurate approximate solution.

2. Algorithm

2.1 Probability Mass Function

Let $X_1, X_2, \ldots, X_n$ be independent Bernoulli random variables where $X_i \sim \text{Bernoulli}(p_i)$. The random variable $Y = \sum_{i=1}^{n} X_i$ follows a PBD with success probability vector $\mathbf{p} = (p_1, p_2, \ldots, p_n)$.

The probability mass function (PMF) of the Poisson Binomial distribution is given by:

$$P(Y = k) = \sum_{A \in F_k} \prod_{i \in A} p_i \prod_{j \in A^c} (1 - p_j),$$

where F_k is the set of all subsets of k integers that can be selected from $\{1, 2, \ldots, n\}$.

2.1.1 Exact Method

To compute the exact PMF efficiently, the Discrete Fourier Transform (DFT) is employed, reducing the computational complexity from $O\left(2^n\right)$ to $O(n \log n)$.

Discrete Fourier Transform

The characteristic function of Y is

$$\phi(t) = E\left(e^{itY}\right) = \prod_{j=1}^{n} \left(1 - p_j + p_j e^{it}\right)$$

Since Y takes values in the finite set $\{0, 1, \ldots, n\}$,

$$\phi(t) = \sum_{k=0}^{n} p_k e^{itk}, \qquad \text{where} \qquad p_k = P(X = k)$$

[1] Hong, Y. (2013). On computing the distribution function for the Poisson binomial distribution.
[2] Hong, Y. (2013). On computing the distribution function for the Poisson binomial distribution.

Choose an integer $m \geq n+1$ and define the discrete frequencies,

$$\omega_r = \frac{2\pi r}{m}, \qquad r = 0, 1, \ldots, m-1$$

Evaluate the characteristic function at each point,

$$\phi(\omega_r) = \sum_{k=0}^{n} p_k e^{\frac{2\pi r}{m} ik}$$

Apply the Inverse Discrete Fourier Transform (IDFT),

$$p_k = \frac{1}{m} \sum_{r=0}^{m-1} \phi(\omega_r) e^{-\frac{2\pi r}{m} ik}, \qquad k = 0, 1, \ldots, m-1$$

Therefore, for every $k = 0, 1, \ldots, n$,

$$P(X = k) = \frac{1}{m} \sum_{r=0}^{m-1} \phi\left(\frac{2\pi r}{m}\right) \exp\left(-\frac{2\pi r}{m} ik\right)$$

where $m \geq n+1$ and $\phi(t) = \prod_{j=1}^{n} (1 - p_j + p_j e^{it})$.

2.1.2 Approximation Method

While the exact DFT-based method reduces computational complexity to $O(n \log n)$, its running time and memory demands become prohibitive for large values of n. Consequently, the Refined Normal Approximation (RNA) is employed to reduce both memory usage and complexity to $O(n)$.

Refined Normal Approximation

Given $\mathbf{p} = (p_1, p_2, \ldots, p_n)$, the mean, variance and skewness can be calculated

$$\mu = \sum_{j=1}^{n} p_j, \qquad \sigma^2 = \sum_{j=1}^{n} p_j(1 - p_j), \qquad \gamma = \frac{1}{\sigma^3} \sum_{j=1}^{n} p_j(1 - p_j)(1 - 2p_j)$$

Apply the RNA using the standard normal $\Phi(z)$ and corresponding PDF $\phi(z)$, including the continuity correction,

$$F(k) \approx \Phi(z) + \frac{\gamma}{6}(1 - z^2)\phi(z), \qquad \text{where} \qquad z = \frac{k + 0.5 - \mu}{\sigma}$$

Therefore, for every $k = 0, 1, \ldots, n$,

$$P_{RNA}(X = k) = F(k) - F(k-1)$$

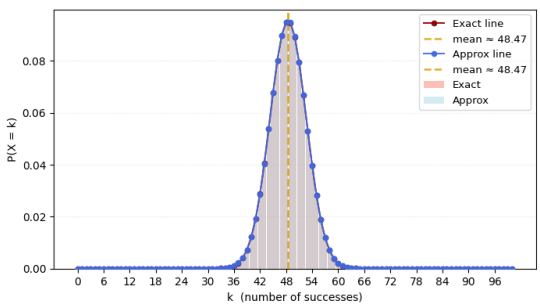
2.1.3 Testing for Different Cases

To study the performance of the algorithms, six different patterns of p_j's are simulated as follows:

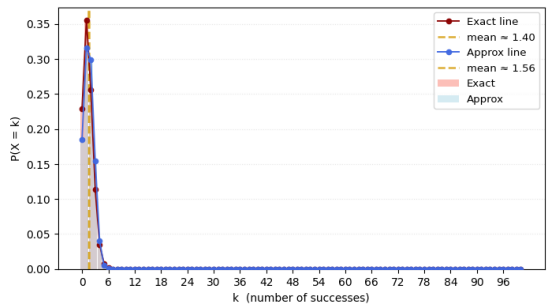
Table 1: Six patterns of p_j's generated

Case	Distribution
1	Uniform$(0, 1)$
2	Beta$(0.1, 3)$
3	Beta$(3, 0.1)$
4	0.5 Beta$(3, 0.1)$ + 0.5 Beta$(0.1, 3)$
5	Beta$(3, 3)$
6	0.5 Beta$(3, 10)$ + 0.5 Beta$(10, 3)$

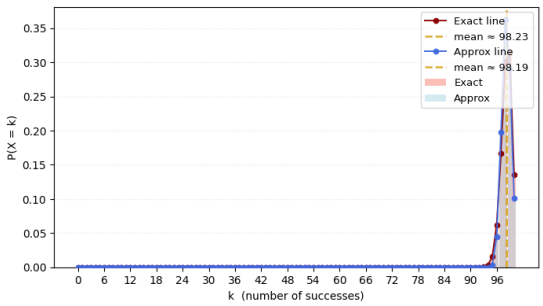
For each case, 100 p_j are generated.



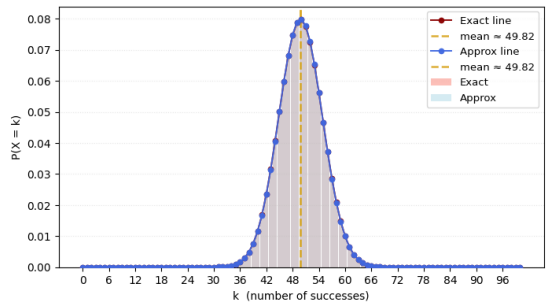
(a) Case 1



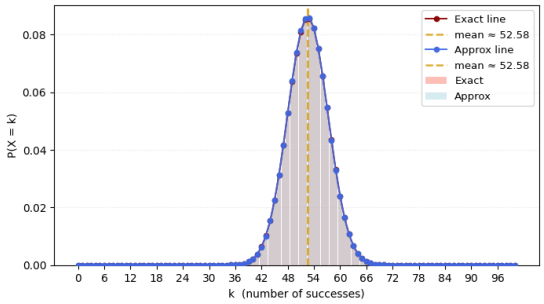
(b) Case 2



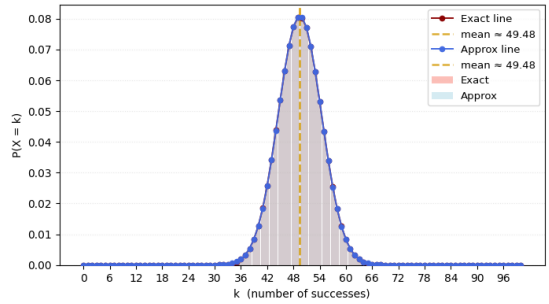
(c) Case 3



(d) Case 4



(e) Case 5



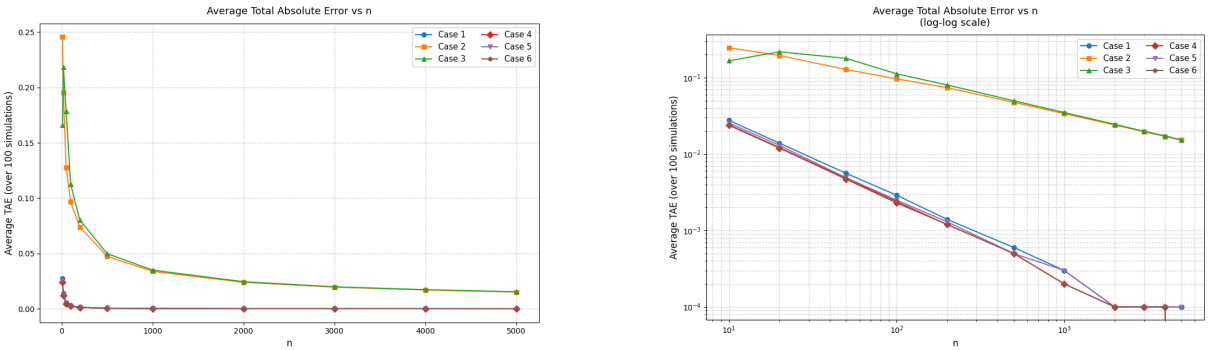
(f) Case 6

Figure 1: PMF Visualization of Exact and Approximation Algorithm for 6 Cases

Table 2: Average TAE for 100 times simulations

n	Case 1	Case 2	Case 3	Case 4	Case 5	Case 6
10	0.0276	0.2456	0.1662	0.0238	0.0255	0.0242
20	0.0139	0.1954	0.2184	0.0120	0.0130	0.0122
50	0.0056	0.1276	0.1787	0.0047	0.0049	0.0048
100	0.0029	0.0965	0.1126	0.0023	0.0025	0.0024
200	0.0014	0.0740	0.0803	0.0012	0.0013	0.0012
500	0.0006	0.0475	0.0499	0.0005	0.0005	0.0005
1000	0.0003	0.0338	0.0350	0.0002	0.0003	0.0002
2000	0.0001	0.0240	0.0245	0.0001	0.0001	0.0001
3000	0.0001	0.0196	0.0199	0.0001	0.0001	0.0001
4000	0.0001	0.0170	0.0173	0.0001	0.0001	0.0001
5000	0.0001	0.0152	0.0154	0.0000	0.0001	0.0000

The average TAE is calculated by $\frac{1}{100}\sum_{i=1}^{100}\sum_{k=0}^{n}|P_i(k) - P_{RNA,i}(k)|$, where i stands for the i-th simulation.



(a) Average TAE

(b) Average TAE in Log Scale

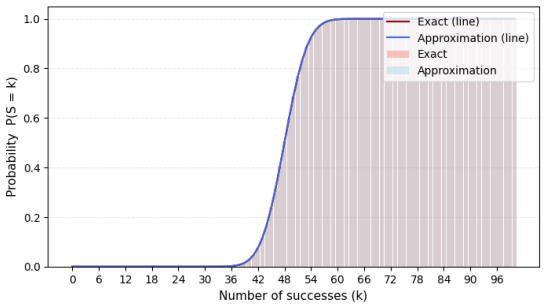
Figure 2: Average TAE Visualization for 6 Cases

Therefore, it can be concluded that across all cases, there is an inverse proportion between the values of n and the average TAE, which indicates that the RNA algorithm is effective for approximating the PMF when n is sufficiently large ($n \geq 2000$). However, for cases involving extreme p_j values (Cases 2 and 3), the convergence rate is substantially slower than in the other cases. In such situations, it is advisable to use the exact DFT algorithm.

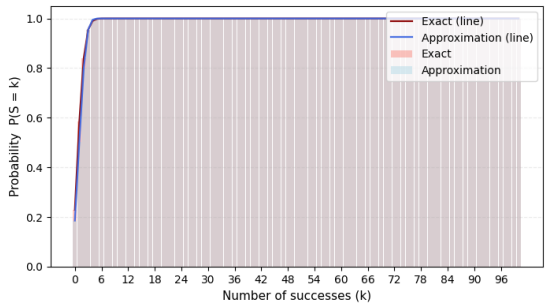
2.2 Cumulative Distribution Function

To be edit.

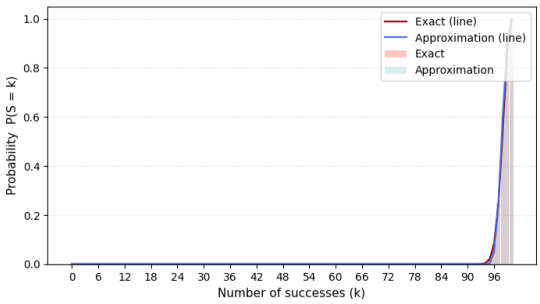
2.2.1 Testing for Different Cases



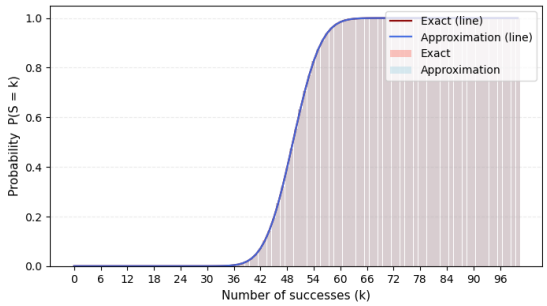
(a) Case 1



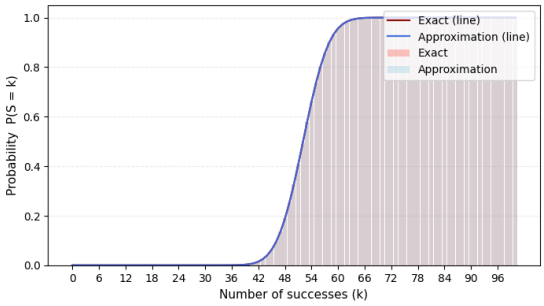
(b) Case 2



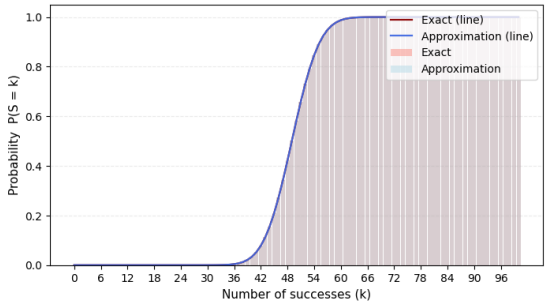
(c) Case 3



(d) Case 4



(e) Case 5



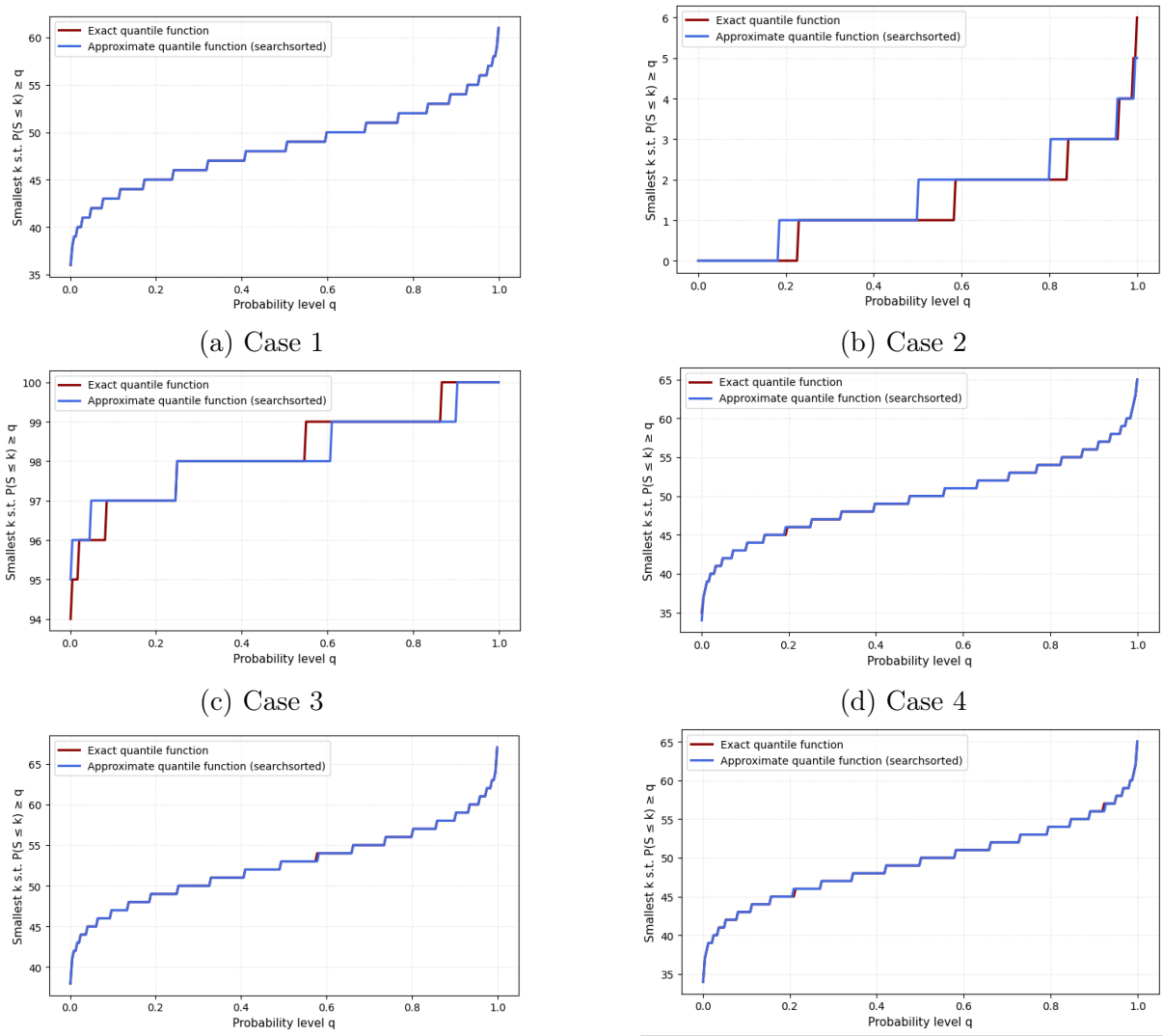
(f) Case 6

Figure 3: CDF Visualization of Exact and Approximation Algorithm for 6 Cases

2.3 Inverse Cumulative Distribution Function

To be edit.

2.3.1 Testing for Different Cases



(a) Case 1

(b) Case 2

(c) Case 3

(d) Case 4

(e) Case 5

(f) Case 6

Figure 4: ICDF Visualization of Exact and Approximation Algorithm for 6 Cases

3. Conclusion

To be edit.